

Les étapes de création d'une Crew d'Agents IA

Résumé méthodologique complet du projet « AI Note Agent »

ÉTAPE 1 : Conception stratégique et définition du scope

Ce que nous avons fait :

- **Définition du problème** : Les notes prises à la volée (réunions, idées, todos, questions) sont désorganisées, pleines d'abréviations et perdent du temps à être reformattées.
 - **Définition de la solution** : Un agent IA autonome qui lit des fichiers `.txt` bruts, les comprend, les structure via API LLM, et génère des documents Word `.docx` professionnels.
 - **Choix de la stack technique** : Python 3.11+, APIs externes (Gemini, Groq), librairie `python-docx` pour la génération de documents, `python-dotenv` pour la sécurité.
 - **Stratégie économique (Token Economy)** : Utiliser plusieurs APIs pour diviser la charge. Groq pour les notes courtes (rapide, gratuit), Gemini pour les notes longues (grand contexte). Cette stratégie vise à ne jamais payer et à optimiser les quotas.
 - **Approche par phases** : Commencer par du Python pur sans framework IA (pour comprendre les fondations), puis éventuellement migrer vers LangChain/LlamaIndex en Phase 4.
-

ÉTAPE 2 : Mise en place de l'environnement de développement

Ce que nous avons fait :

2.1 Structure de dossiers modulaire

Création d'une architecture propre dès le départ :

```
my-agent/
├── .env                # Clés API (jamais commité)
├── .gitignore
├── requirements.txt
├── main.py             # Point d'entrée
├── notes/              # Input : fichiers .txt bruts
├── output/             # Output : fichiers .docx générés
├── templates/          # Prompts JSON par type de note
├── src/                # Code source modulaire
│   ├── config.py
│   ├── apis/           # Wrappers API (Groq, Gemini)
│   ├── classifieur/    # Détection du type de note
│   ├── extracteurs/    # Construction des prompts
│   ├── renderers/      # Générateurs de docx
│   └── validator.py    # Vérification des réponses
```

2.2 Gestion de l'environnement virtuel (`venv`)

- Création : `python -m venv venv`
- Activation Windows PowerShell : `.\venv\Scripts\Activate.ps1`
- Installation des dépendances : `pip install python-dotenv groq python-docx`
- Sauvegarde : `pip freeze > requirements.txt`

2.3 Sécurisation des secrets

- Création d'un fichier `.env` contenant `GROQ_API_KEY=` et `GEMINI_API_KEY=`
- Ajout de `.env` et `venv/` dans `.gitignore` pour ne jamais les pousser sur Git
- Création de `src/config.py` qui charge les variables d'environnement via `load_dotenv()`

ÉTAPE 3 : Phase de validation et débannage des APIs

Ce que nous avons fait :

3.1 Création d'un script de test API

- Fichier `test_apis.py` pour vérifier que les deux clés fonctionnent avant de coder quoi que ce soit.
- Le script teste Gemini et Groq avec un prompt simple : « Say X is working ».

3.2 Résolution des problèmes rencontrés (en pratique)

- **Erreur de frappe** : Installation de `groc` au lieu de `groq` (package aléatoire qui ne compile pas). Correction : `pip install groq`.
- **Package Gemini obsolète** : `google-generativeai` est déprécié. Migration vers `google-genai` (nouveau SDK officiel).
- **Modèles décommissionnés** : `gemini-1.5-flash` et `llama3-8b-8192` n'existent plus. Mise à jour vers `gemini-2.0-flash` et `llama-3.1-8b-instant`.
- **Blocage quota Gemini (429 RESOURCE_EXHAUSTED)** : Malgré 0 utilisation réelle apparente, le compte free tier Google avait un quota à 0 (problème de région/billing). Décision stratégique : **pivot immédiat vers Groq uniquement** pour la V1, et réintégration de Gemini plus tard.

ÉTAPE 4 : Construction du pipeline central (Phase 1 — MVP)

Ce que nous avons fait : Un agent fonctionnel de bout en bout pour un seul type de note (Meeting).

4.1 Couche API — Connexion à Groq

- Fichier `src/apis/groq.py` :
 - Création d'un client Groq réutilisable avec la clé `.env`
 - Fonction `call_groq(prompt, retries=3)` avec **retry logic** :
 - `ConnectionError` → attente exponentielle (1s, 2s, 4s) puis retry
 - `ValueError` / erreur JSON → arrêt immédiat
 - Exception inconnue → relance (`raise`)

- Prompt système imposant : « Always respond with valid JSON only. No markdown. »
- Paramètres : `temperature=0.2` (faible créativité pour du JSON fiable),
`max_tokens=1024`

4.2 Couche Extraction — Templates de prompts

- Fichier `templates/meeting.json` :
 - Contient un prompt détaillé demandant à l'IA d'extraire : `title`, `date`,
`participants`, `agenda_items`, `decisions_made`, `action_items`, `summary`, `tags`
 - Liste des `required_fields` pour validation ultérieure
- Fichier `src/extractors/prompt_builder.py` :
 - Charge le template JSON
 - Injecte la note brute dans le placeholder `{raw_note}`
 - Retourne le prompt complet + les champs obligatoires

4.3 Couche Rendu — Génération de documents Word

- Fichier `src/renderers/meeting.py` :
 - Utilisation de `python-docx` pour créer un `.docx` structuré
 - Titre centré, méta-informations (date, participants), listes à puces pour l'agenda, décisions en vert, action items numérotées avec propriétaire et deadline, résumé, tags
 - Création automatique du dossier `output/` si inexistant

4.4 Point d'entrée — Orchestration

- Fichier `main.py` :
 1. Liste les fichiers `.txt` dans `notes/`
 2. Lit le contenu brut
 3. Charge le template `meeting`
 4. Appelle Groq

5. Parse la réponse JSON (avec nettoyage des fences markdown ```json)
6. Valide les champs
7. Appelle le renderer pour générer le `.docx` dans `output/`

4.5 Test concret

- Création de `notes/meeting_sample.txt` avec un texte désorganisé
- Exécution : `python main.py`
- Résultat : `output/meeting_sample.docx` formaté et ouvrable dans Word

ÉTAPE 5 : Ajout de l'intelligence multi-types (Phase 2)

Ce que nous avons fait : L'agent détecte automatiquement le type de note et adapte son traitement.

5.1 Système de classification double couche

- **Classifieur heuristique** (`src/classifier/heuristic.py`) :
 - Analyse les mots-clés dans le texte brut (`meeting` , `todo` , `idea` , `question` , etc.)
 - Attribue un score pondéré et une confiance (0.0 à 1.0)
 - Coût : **zéro token API**, exécution instantanée
- **Classifieur API fallback** (`src/classifier/api_classifier.py`) :
 - Si la confiance heuristique est < 0.3 ou type « unknown », on envoie un mini-prompt à Groq pour classifier
 - Retourne un JSON `{"type": "...", "confidence": ...}`

5.2 Création des templates métier

Ajout de 3 nouveaux templates JSON :

- `idea.json` : `problem_statement` , `proposed_solution` , `potential_impact` , `required_resources` , `next_steps`

- `todo.json` : `tasks` (objets avec `description`, `priority`, `due_date`, `status`), `categories`
- `qa.json` : `question`, `answer`, `sources`, `confidence`, `follow_up_questions`

5.3 Création des renderers spécialisés

- `renderers/idea.py` : Mise en page créative avec sections Problème/Solution/Impact, solution en bleu gras, ressources en puces
- `renderers/todo.py` : Tableau Word à 4 colonnes (Task, Priority, Due Date, Status) avec codes couleur (rouge/orange/vert)
- `renderers/qa.py` : Badge de confiance coloré, question en bleu gras, réponse structurée, sources et questions de suivi

5.4 Validation et robustesse

- Fichier `src/validator.py` :
 - Vérifie que tous les `required_fields` sont présents
 - Remplit les champs manquants avec des valeurs par défaut (`"N/A"`, `[]`, `null`)
 - Force la conversion en liste si un champ attendu est mal typé

5.5 Orchestration mise à jour

- `main.py` modifié pour :
 1. Classifier la note (heuristique → fallback API)
 2. Charger le bon template dynamiquement
 3. Sélectionner le bon renderer via un dictionnaire `RENDERERS = {"meeting": ..., "idea": ...}`
 4. Nommer le fichier de sortie avec le type détecté (`meeting_sample_meeting.docx`)

ÉTAPE 6 : Extension vers des types avancés (Question-Only & Default)

Ce que nous avons fait : Gérer les cas limites et les besoins spécifiques.

6.1 Type « question_only »

- **Différence clé :** Contrairement au type `qa` (qui structure une Q&A déjà existante dans la note), ce type envoie une **question brute sans réponse** à Groq et demande à l'IA de **générer la réponse elle-même**.
- Template demandant : `title` , `question` , `answer` , `confidence` , `sources` , `related_topics`
- Renderer dédié avec mise en page Q&A professionnelle

6.2 Type « default » (Fallback intelligent)

- Comportement : Quand le texte ne correspond à aucun type (notes mixtes, brouillons, phrases désorganisées), l'agent passe en mode « recherche générale ».
- L'IA analyse le texte, répond aux questions implicites, définit les termes clés, et propose des actions.
- Champs : `summary` , `questions_answered` , `key_terms` , `insights` , `suggested_actions`
- Ce type sert aussi de **filet de sécurité** si le classifieur échoue.

6.3 Mise à jour transversale

- Ajout des mots-clés `question_only` et `default` dans le classifieur heuristique
- Mise à jour du prompt du classifieur API pour accepter 6 types au lieu de 4
- Extension du validateur pour gérer les nouvelles listes (`questions_answered` , `key_terms` , etc.)
- Mise à jour du dictionnaire `RENDERERS` dans `main.py`

ÉTAPE 7 : Comprendre la mécanique fondamentale API ↔ Python ↔ Document

Ce que nous avons clarifié :

7.1 Le flux de données exact

```
Fichier .txt brut (string)
↓
Prompt construit (template + note injectée)
↓
Appel API Groq → retourne une STRING JSON
↓
Nettoyage des fences markdown (```json ... ```)
↓
json.loads() → Python dict
↓
Renderer python-docx → Document .docx
```

7.2 Frontière des responsabilités

- **L'API (Groq) fait** : La compréhension sémantique, l'extraction d'entités, la structuration logique
- **Le code Python fait** : Le parsing, la validation, la mise en page visuelle (couleurs, tableaux, polices), la gestion des fichiers

7.3 Gestion des images (discutée et planifiée)

- **Stratégie 1** : Gemini Vision (envoi texte + image en un seul appel) — bloqué pour l'instant à cause des quotas
- **Stratégie 2** : OCR avec `pytesseract` pour extraire le texte des images et l'ajouter à la note avant envoi à Groq
- **Stratégie 3** : Découverte automatique des images associées (même nom de fichier, extensions `.png` / `.jpg`)

ÉTAPE 8 : Collaboration d'équipe et versioning

Ce que nous avons fait :

8.1 Mise sous Git

- `git init` dans le dossier projet
- `.gitignore` configuré pour exclure `.env`, `venv/`, `__pycache__/`, `output/`
- Commit initial : `git commit -m "Phase 1: Core pipeline..."`

- Push vers un dépôt GitHub distant

8.2 Guide d'onboarding pour les coéquipiers

Création d'une procédure claire pour que chaque membre puisse rejoindre :

1. Créer compte GitHub et cloner le repo
2. Installer Git localement
3. Créer son propre `venv` et activer
4. `pip install -r requirements.txt`
5. Créer son propre fichier `.env` avec sa clé Groq personnelle
6. Tester avec `python test_apis.py`
7. Ajouter ses notes dans `notes/` et lancer `python main.py`

8.3 Répartition modulaire du travail

- Chaque membre peut posséder un module (`apis/`, `renderers/`, `classifier/`) sans conflit
- Le système de templates permet d'ajouter un nouveau type de note sans toucher au code existant

ÉTAPE 9 : Industrialisation et roadmap future

Ce que nous avons planifié :

9.1 Passage en application web

- **Backend** : API REST avec Flask ou FastAPI (endpoints `POST /process-note`, retour JSON + lien de téléchargement)
- **Frontend** : Interface web avec upload de fichiers, zone de texte, sélecteur de template, prévisualisation du JSON, bouton de téléchargement du `.docx`
- **Fonctionnalités** : Historique des traitements, override manuel du type détecté, barre de progression

9.2 Réintégration du multi-API

- Réactivation de Gemini pour le **smart routing** (notes < 100 mots → Groq, notes longues → Gemini)
- Système de fallback si une API est en panne ou rate-limitée

9.3 Présentation académique

- Structure de 12 slides définie : Titre → Problématique → Architecture → Workflow → Détails par étape → Outils → Conclusion → Q&A
- Prompt détaillé rédigé en français pour générer automatiquement le contenu de la présentation
- Emplacements réservés pour les images (architecture, flowchart, captures d'écran avant/après)

Synthèse : La philosophie de la Crew en action

Ce que ce projet illustre sur la création d'une **Crew d'Agents IA** :

Pilier	Application concrète dans notre projet
Modularité	Chaque type de note = un agent spécialisé (template + renderer)
Classification	Un « chef d'orchestre » (classifier) route le travail vers le bon agent
Tolérance aux pannes	Retry logic, fallback API, type « default » si aucun agent ne correspond
Économie des ressources	Heuristique gratuite avant appel API, choix stratégique de Groq
Extensibilité	Ajouter un nouveau type = créer 2 fichiers (template JSON + renderer) sans toucher au reste
Séparation des préoccupations	L'IA fait le cerveau (compréhension), Python fait les mains (formatage)